

## PAPER

# Removing a Redundant Tool-Call Rewrite in Multi-Agent RAG: A Checked Direct-Dispatch Adapter Evaluated from the Call Boundary to Concurrency 100

**SUMMARY** In a multi-agent retrieval-augmented generation (RAG) system for legal search, a preparation agent already forms the complete arguments of a search tool, but a second language-model call rewrites them into a tool call before the tool runs. This paper removes that rewrite with a checked direct-dispatch adapter that validates the prepared arguments and calls the same handler, optionally inside a signed execution envelope. Branch traces show that the rewrite never changed the prepared arguments in 648 of 648 invocations, so removing it is safe. Before the change, the rewrite cost 0.54 s per search branch at concurrency 4 and grew to 22 s per branch at concurrency 100 because it waited in the tool engine's queue; it accounted for 41% of that engine's requests. After the change, the boundary costs 0.5–0.9 ms, the tool engine's queue time per request falls by 29–72%, and the time from request start to search completion falls by 0.46–0.64 s in every round. The end-to-end response-time gain is smaller than the removed work: 0.5% at concurrency 4, 1.4–2.9% at 8–16, and 2.3–2.6% at 64–100, with per-question sign tests favoring the adapter at every load except 32, because the orchestrator engine saturates from concurrency 32 and absorbs the released time. Source-set agreement with the original path is within the retrieval path's own instance-to-instance variation, so no quality loss is attributable to the adapter.

**key words:** *multi-agent RAG, tool calling, direct dispatch, redundant model call, load testing, retrieval agreement*

## 1. Introduction

Retrieval-augmented generation (RAG) combines document retrieval with language generation[1]. A multi-agent RAG system divides one request among agents for question analysis, search preparation, search, evidence selection, and answer writing. Each hand-off between agents can cost a language-model call, and not every call makes a decision. Some only reformat data that the program already holds.

This paper studies one such call in a production multi-country legal search system. A preparation agent produces the country, search terms, and keywords of a search. The original implementation then hands these values to a tool-calling model, which emits the same values as a structured tool call, and only then does the search handler run. The second call chooses no tool and adds no term. It is a text-to-JSON relay between two parts of one program.

The relay is not free. At concurrency 4 it costs 0.54 s per search branch. Under load it waits in the queue of the tool-serving engine like every other request, so its cost grows to 4.5 s per branch at concurrency 16 and 22 s at concurrency 100, and it makes up 41% of all requests served by that engine.

We remove the relay with a direct-dispatch adapter. Trusted application code registers the search operation

with an argument validator, copies the prepared arguments through canonical JSON, validates them, and calls the existing handler; the result enters the existing parser unchanged. An optional signed envelope adds scope, expiry, and one-time-use checks for deployments with an untrusted hop. We verify the feature before measuring it: prepared and executed arguments are hashed on every branch, and the rewrite is shown to preserve them in all 648 recorded invocations.

The evaluation asks what the removal buys at three scopes and across load. At the boundary, the rewrite time disappears at every concurrency. At the search-completion endpoint, requests reach the end of their search 0.46–0.64 s earlier. At the user-visible response, the gain is 0.5% at concurrency 4 and 1.4–2.9% below saturation, with per-question sign tests favoring the adapter at every level except 32. The reason the whole-request gain is small is measured, not assumed: the orchestrator engine saturates from concurrency 32 and its queue absorbs the time released from the search branch. Quality is checked against the original path with instance-level controls, and no source-set loss is attributable to dispatch.

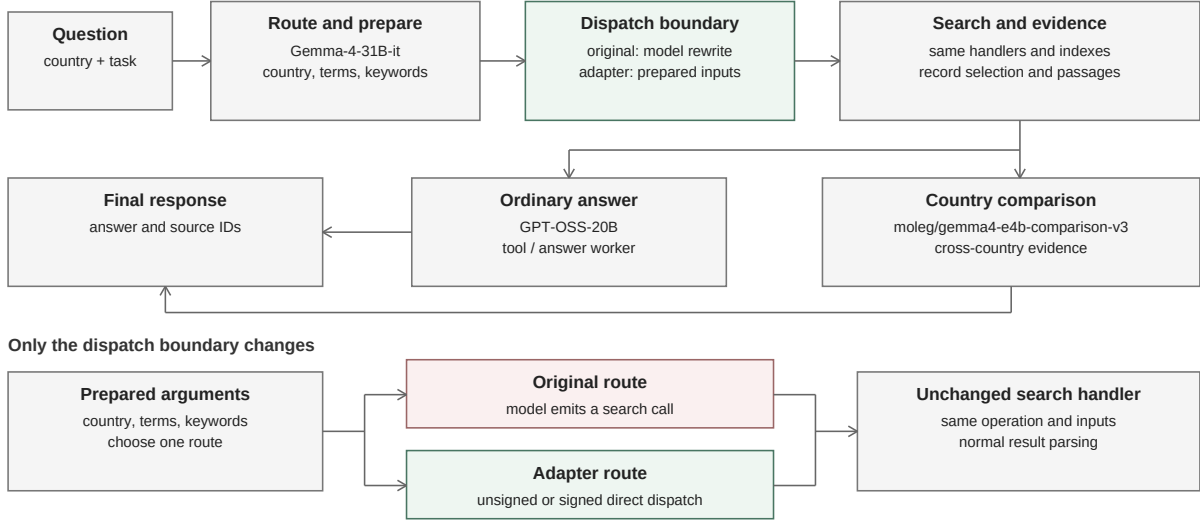
## 2. Related Work

RAG supports generation with retrieved documents[1]. ReAct interleaves reasoning and actions[2]; Toolformer learns when to call an API[3]; ToolLLM studies tool use over a large API collection[4]. These works decide which tool to call. Our change applies after that decision, when application code already holds the operation and its arguments.

AutoGen structures agent conversations[5]. LLMCompiler separates planning from execution so that a program dispatches ready function calls[6]; its executor is the closest analogue to our adapter, which executes one prepared call without a planner. Parrot and Agentix improve serving by exposing application structure to the engine[7], [8]; SGLang reuses KV state and speeds structured decoding[9]; RAGCache caches knowledge states[10]. Those approaches change the serving layer. We leave the engines unchanged and remove one request from the application.

HMAC authenticates a message with a shared secret[11]; Macaroons extend it with contextual caveats[12]. Our optional envelope uses HMAC-SHA256 with a scope and a nonce as a deployment safeguard. It is not a new authorization design and, as shown below, not the source of any speedup.

## Search workflow and the changed dispatch step



Envelope checks stay inside one trusted process; they are safeguards, not inference acceleration.

Existing safety service: Kanana-safeguard-8b. It is unchanged and outside the replaced step.

**Fig. 1** The search workflow and the changed step. In the original route a model rewrites the prepared arguments into a tool call; in the adapter route the same arguments are validated and dispatched directly to the same handler.

### 3. The Existing Workflow and Its Redundant Call

#### 3.1 System

The system serves country-specific legal questions. An orchestrator model classifies the question, prepares search terms and keywords, and later selects evidence; a tool/answer worker runs tool-call loops and writes ordinary answers; a separate comparison model writes cross-country comparisons; a safety model screens requests. Table 1 lists the served models. Two NVIDIA H200 NVL GPUs serve all of them through vLLM. The search handlers query 3,194,266 original-law chunks, 454,351 translated-law chunks, and 19,583 title records.

**Table 1** Model roles in the tested application.

| Component and model                                   | Work in the search workflow                                       |
|-------------------------------------------------------|-------------------------------------------------------------------|
| Orchestrator:<br>Gemma-4-31B-it                       | Classification, search preparation, evidence selection, fallbacks |
| Tool / answer:<br>GPT-OSS-20B                         | Search-call rewriting, tool loops, ordinary answers               |
| Country comparison:<br>moleg/gemma4-e4b-comparison-v3 | Cross-country comparison text                                     |
| Safety:<br>Kanana-safeguard-8b                        | Request safety checks                                             |
| Search handlers                                       | Text and title indexes                                            |

#### 3.2 The rewrite step

The search-dispatch function receives the country, merged keywords, search terms, and a request ID. The first three form the argument object of the registered search tool. In the original route the function sends this object to the tool/answer model together with the tool schema, receives the model's tool call, and runs the handler with the returned arguments. Source inspection shows that the loop allows one model round, keeps the raw handler result, and passes it to the parser; the model can only reproduce or alter the arguments.

#### 3.3 What the step costs

Table 2 quantifies the step in the original route, measured with the branch traces of Section 5. At concurrency 4 the rewrite adds 543 ms to a search branch whose handler takes 575 ms. Under load the rewrite is queued behind other requests of the tool/answer engine. Its mean cost per branch rises to 1.2 s at concurrency 8, 4.5 s at 16, 6.4 s at 32, 19.1 s at 64, and 22.4 s at 100, with 95th percentiles of 16–32 s from concurrency 16 upward. Because every search branch issues one rewrite, the engine serves 500–521 requests per 200-question batch of which 216 are rewrites, and its queue depth grows to 143 waiting requests at concurrency 100.

#### 3.4 Why the step is redundant

A fixed operation does not by itself make the rewrite unnec-

**Table 2** Cost of the rewrite in the original route per 200-question batch. Rewrite time is the non-handler time of a search branch; queue time is the tool/answer engine’s summed queue time divided by 200 requests; waiting is the batch maximum.

| $C$ | Rewrite mean (s) | Rewrite p95 (s) | Handler (s) | Engine requests | Queue per request (s) / max waiting |
|-----|------------------|-----------------|-------------|-----------------|-------------------------------------|
| 4   | 0.54             | –               | 0.58        | 500             | 0.05 / 1                            |
| 8   | 1.2              | 4.3             | 0.61        | 517             | 1.19 / 7                            |
| 16  | 4.5              | 16.2            | 0.57        | 521             | 8.44 / 17                           |
| 32  | 6.4              | 22.0            | 0.69        | 521             | 12.11 / 39                          |
| 64  | 19.1             | 32.0            | 2.80        | 508             | 33.72 / 91                          |
| 100 | 22.4             | 29.0            | 5.00        | 456             | 39.04 / 143                         |

essary; the model might correct an argument. We therefore hash the prepared object and the object the handler actually receives on every search branch. Across the three-round study of Section 5, all 648 rewrites in the original route produced arguments identical to the prepared ones; no changed field was recorded. The rewrite is a relay with no observed effect on what is executed.

## 4. The Direct-Dispatch Adapter

### 4.1 Execution boundary

The adapter registers a read-only operation, an argument validator, and an asynchronous handler. For legal search the validator requires exactly three fields: a nonempty string of search terms, a country string or null, and a list of string keywords. Dispatch copies the prepared object through JSON serialization with fixed key order and separators, validates the copy, and calls the registered handler. The handler’s result enters the existing parser and the existing return structure, so later agents receive the same fields as before.

Eligibility is decided before dispatch. If the object fails the validator, the integration takes the original route. After dispatch, a failure raises an error; it is never silently retried through a model. The adapter contains no learned routing and no scheduling; its only effect on a request is one omitted model call per eligible search branch.

### 4.2 Optional signed envelope

For deployments where the plan crosses an untrusted hop, dispatch can wrap the object in an envelope carrying a version, receiver, request scope, operation, issue time, expiry, and a 128-bit nonce, authenticated by

$$M = \text{HMAC-SHA256}(K, P), \quad (1)$$

where  $P$  is the canonical payload and  $K$  a per-invocation 32-byte key. The executor checks the code, version, receiver, scope, time range, registered operation, argument schema, and nonce, and marks the nonce spent before calling the handler. In the tested prototype issuer and executor share one process, so the envelope is a safeguard and the unsigned direct call is sufficient; both are evaluated. A 200-case contract test (40 valid plans and 40 each of altered, wrong-scope, expired, and replayed plans) produced the expected decision

in every case.

### 4.3 Feature verification

Before any latency measurement, the adapter was checked on the same traces. In each of 600 attempted questions per condition, all 216 search branches (132 single-search questions, 42 two-search comparison questions, 26 questions that bypass search) reached the handler once; no schema fallback and no boundary error occurred. Prepared and executed argument hashes were identical on all 648 branches of both adapter arms. Table 3 gives the boundary timing at concurrency 4: the non-handler time falls from 543.140 ms to 0.521 ms for unsigned dispatch and 0.880 ms for signed dispatch, while handler time is unchanged within 12 ms. Signing costs 0.36 ms per branch.

**Table 3** Search-branch timing at concurrency 4, pooled over 648 branches per condition (ms). Non-handler time is the full boundary minus the nested handler.

| Interval               | Original | Unsigned | Signed  |
|------------------------|----------|----------|---------|
| Full boundary          | 1118.451 | 588.066  | 578.052 |
| Search handler         | 575.311  | 587.545  | 577.173 |
| Non-handler            | 543.140  | 0.521    | 0.880   |
| Setup / key            | –        | –        | 0.083   |
| Issue                  | –        | –        | 0.249   |
| Copy and validate      | –        | 0.070    | –       |
| Verify / handler entry | –        | 0.052    | 0.144   |
| Parsing and tracing    | –        | 0.399    | 0.404   |

## 5. Experimental Method

### 5.1 Conditions and runs

Three conditions share one frozen application image, the same models, prompts, tool handlers, search data, and vLLM options: the original route, unsigned direct dispatch, and signed direct dispatch. Each condition runs in its own isolated application instance (2 CPU cores, 8 GiB) on the GPU host; a separate client instance (2 cores, 2 GiB) generates load. Production traffic shares the engines.

Three runs are reported. (1) Concurrency 4: 200 country-specific legal questions per condition in three rounds whose serial order rotates (a  $3 \times 3$  Latin square), 1,800 requests, request timeout 180 s. (2) Load sweep: original

versus signed dispatch at concurrency 8, 16, 32, 64, and 100, two rounds per level with the order reversed, 4,000 requests, timeout 600 s, and the instance's admission limit raised from 4 to 100 so that client concurrency reaches the engines. (3) Instance control: two instances of the original route and two of signed dispatch, one batch each at concurrency 4, with handler-output hashes recorded. The question set has 20 countries and four types of 50 questions (finding a law, explaining content, comparing countries, analyzing a case), with a fixed seeded order. All 6,600 requests completed; six read timeouts occurred, all in the concurrency-4 runs.

Before each batch the runner waited for idle engines. In the load sweep, after ten cells, one request that never completed remained running in the orchestrator engine; the remaining ten cells and the instance control used a gate that tolerates one running request per engine with zero waiting, applied equally to all arms.

## 5.2 Measurements

Every request stores its client start and end time, status, and returned law IDs. Application traces record every wrapped model call with token counts and API time, and every search branch with prepared and executed argument hashes, handler time, and boundary time. vLLM counters of each engine are sampled every second; batch differences give requests, queue time, prefill and decode time, and preemptions, and gauges give waiting depth and KV usage. A traffic check compares engine request counts with wrapped calls: in every batch the orchestrator engine's count equals the wrapped orchestrator calls within one request and the safety engine serves exactly 200, so no outside traffic reached the engines during measurement.

For a completed pair of the same question in two conditions,  $d_i$  is the original minus the candidate response time; positive values favor the adapter. We report the mean with a 10,000-sample question-bootstrap interval, the median, and an exact two-sided sign test. For the sweep, each question's two round differences are averaged before pooling. The same statistics are applied to original-versus-original pairs across rounds as a null control. Two intermediate endpoints use the traces:  $E_i$ , from client start to the last search-branch completion, and  $S_i$ , from the first prepared-search event to that completion. Source agreement compares deduplicated law-ID sets: nonempty-reference recall, exact-set agreement, and within-arm agreement across rounds or instances.

## 6. Results

### 6.1 The rewrite is removed at every load

Table 4 extends Table 2 to the adapter. At every concurrency the adapter's non-handler time stays at 1.2–1.6 ms while the original rewrite costs 1.2–22.4 s. The tool/answer engine serves 41–46% fewer requests per batch (305, 306, 301, 286, and 246 against 517, 521, 521, 508, and 456), its queue

time per request falls by 47%, 29%, 68%, 57%, and 72% at concurrency 8, 16, 32, 64, and 100, and its maximum waiting depth falls from 7–143 to 4–21 requests. At concurrency 4 the engine's summed queue time per batch falls from 9.2 s to below 0.01 s, and its prompt tokens by only 1.4–2.3%: the rewrite is request-heavy but token-light, about 105–165 prompt tokens per call.

### 6.2 The saving reaches the search-completion endpoint

At concurrency 4, signed dispatch shortens the interval from the first prepared search to the last search completion by 0.543, 0.504, and 0.561 s in the three rounds, with question-bootstrap intervals of [0.511, 0.577], [0.483, 0.526], and [0.522, 0.606], and the interval from client start to the last search completion by 0.529, 0.639, and 0.464 s, all with intervals above zero. Unsigned dispatch gives 0.523, 0.497, and 0.550 s on the search interval. The same endpoints between two original rounds change by at most 0.06 s (search interval) and 0.12 s (request start), so the reduction is an order of magnitude above ordinary variation. The removed call therefore reaches the request's path: requests finish searching about half a second earlier at concurrency 4, and by the amounts of Table 4 under load.

### 6.3 End-to-end response time

#### (1) Concurrency 4.

Table 5 gives the paired response time by round. Signed dispatch changes the round mean by  $-5.42\%$ ,  $+0.80\%$ , and  $+5.25\%$  and unsigned dispatch by  $-8.52\%$ ,  $+1.00\%$ , and  $+4.94\%$ ; the two original rounds differ from each other by  $-1.526$ ,  $-1.019$ , and  $+0.546$  s. On the 195 questions completed in all nine batches, the position-balanced estimate is a 0.123-s (0.50%) decrease for signed dispatch with interval  $[-1.153, 1.290]$  and a 0.077-s increase for unsigned dispatch. Medians are positive in every signed round (0.022, 0.226, 0.556 s) and sign tests favor unsigned dispatch in rounds 2 and 3 ( $p = 0.0002$  and  $0.0054$ ) and signed dispatch in round 3 ( $p = 0.013$ ). The paired standard deviation is 11–16 s, so a 0.6-s mean effect is below what 200 pairs can resolve; the null control shows the same size of round-to-round movement.

#### (2) Concurrency 8 to 100.

Table 6 gives the rotated sweep. Pooled savings are 0.44 s (1.4%) at 8, 1.54 s (2.9%) at 16,  $-3.35$  s ( $-3.6\%$ ) at 32, 3.54 s (2.6%) at 64, and 3.73 s (2.3%) at 100. Per-question sign tests favor signed dispatch at 8, 16, 64, and 100 (up to 149 of 200 questions faster at 100), both rounds have positive medians at 16, 64, and 100, and the pooled mean interval excludes zero at 100. Concurrency 32 is the least stable level: the two rounds disagree by 9 s and the arm that ran second was slower in both.

**Table 4** Load sweep, original / signed dispatch, mean of two rounds per level. Rewrite time is the original route’s non-handler time per branch; the signed adapter’s non-handler time is 1.2–1.6 ms at every level. Queue times are per-batch sums divided by 200 requests; waiting and KV are batch maxima.

| C   | Rewrite mean / p95 (s) | Handler (s) | Worker requests | Worker queue/req. (s) | Worker max wait | Orch. queue/req. (s) | Orch. max wait | Orch. max KV |
|-----|------------------------|-------------|-----------------|-----------------------|-----------------|----------------------|----------------|--------------|
| 8   | 1.2 / 4.3              | 0.61 / 0.62 | 517 / 305       | 1.19 / 0.63           | 7 / 4           | 0.14 / 0.05          | 4 / 4          | 0.65 / 0.60  |
| 16  | 4.5 / 16.2             | 0.57 / 0.68 | 521 / 306       | 8.44 / 6.03           | 17 / 9          | 0.45 / 0.36          | 5 / 6          | 0.91 / 0.95  |
| 32  | 6.4 / 22.0             | 0.69 / 0.84 | 521 / 301       | 12.11 / 3.87          | 39 / 10         | 28.4 / 29.4          | 29 / 26        | 1.00 / 1.00  |
| 64  | 19.1 / 32.0            | 2.80 / 3.27 | 508 / 286       | 33.72 / 14.63         | 91 / 20         | 86.9 / 84.6          | 78 / 67        | 1.00 / 1.00  |
| 100 | 22.4 / 29.0            | 5.00 / 5.97 | 456 / 246       | 39.04 / 10.75         | 143 / 21        | 121.7 / 137.9        | 136 / 148      | 1.00 / 1.00  |

**Table 5** Completed-pair response time at concurrency 4 by round (200 questions per batch). Saving is original minus candidate. Intervals are question-bootstrap percentiles. The last three rows are the null control between original rounds.

| Round | Pair                 | Pairs | Original (s) | Candidate (s) | Saving (s) | 95% interval (s) | Median (s) | Faster/slower (sign <i>p</i> ) |
|-------|----------------------|-------|--------------|---------------|------------|------------------|------------|--------------------------------|
| 1     | original vs unsigned | 198   | 23.733       | 25.755        | −2.022     | [−5.364, 0.773]  | 0.267      | 107/91 (0.286)                 |
| 1     | original vs signed   | 199   | 23.677       | 24.960        | −1.283     | [−3.145, 0.354]  | 0.022      | 100/99 (1.000)                 |
| 2     | original vs unsigned | 200   | 25.151       | 24.901        | 0.251      | [−2.076, 2.308]  | 1.160      | 127/73 (0.0002)                |
| 2     | original vs signed   | 200   | 25.151       | 24.950        | 0.201      | [−2.242, 2.061]  | 0.226      | 110/90 (0.179)                 |
| 3     | original vs unsigned | 198   | 24.706       | 23.487        | 1.220      | [−0.662, 3.111]  | 0.601      | 119/79 (0.0054)                |
| 3     | original vs signed   | 198   | 24.687       | 23.391        | 1.296      | [−0.543, 3.288]  | 0.556      | 117/81 (0.013)                 |
| 1, 2  | original vs original | 199   | 23.677       | 25.203        | −1.526     | [−3.446, 0.247]  | −0.160     | 95/104 (0.571)                 |
| 1, 3  | original vs original | 198   | 23.685       | 24.704        | −1.019     | [−3.032, 0.816]  | −0.099     | 95/103 (0.619)                 |
| 2, 3  | original vs original | 199   | 25.197       | 24.651        | 0.546      | [−0.970, 2.037]  | 0.076      | 102/97 (0.777)                 |

**Table 6** Load sweep: completed-pair response time, original versus signed dispatch, 200 pairs per cell. Pooled rows average each question’s two round differences; their intervals are question-bootstrap percentiles and their sign tests use the per-question averages.

| C   | Round (first arm) | Original (s) | Signed (s) | Saving (s)    | 95% interval (s) | Median (s) | Faster/slower | Sign <i>p</i> |
|-----|-------------------|--------------|------------|---------------|------------------|------------|---------------|---------------|
| 8   | 1 (original)      | 32.23        | 32.52      | −0.29         | [−3.41, 2.14]    | 0.81       | 117/83        | 0.019         |
| 8   | 2 (signed)        | 32.25        | 31.08      | 1.18          | [−0.97, 3.21]    | 0.31       | 105/95        | 0.525         |
| 8   | pooled            | 32.24        | 31.80      | 0.44 (1.4%)   | [−1.44, 2.16]    | −          | 117/83        | 0.019         |
| 16  | 1 (original)      | 52.81        | 52.18      | 0.63          | [−2.48, 3.41]    | 2.85       | 121/79        | 0.004         |
| 16  | 2 (signed)        | 54.68        | 52.22      | 2.46          | [−0.69, 5.64]    | 2.72       | 118/82        | 0.013         |
| 16  | pooled            | 53.75        | 52.20      | 1.54 (2.9%)   | [−0.97, 3.98]    | −          | 120/80        | 0.006         |
| 32  | 1 (original)      | 90.13        | 98.11      | −7.98         | [−11.36, −4.69]  | −3.42      | 66/134        | < 0.001       |
| 32  | 2 (signed)        | 93.81        | 92.54      | 1.28          | [−2.74, 5.22]    | −0.14      | 99/101        | 0.944         |
| 32  | pooled            | 91.97        | 95.32      | −3.35 (−3.6%) | [−6.14, −0.67]   | −          | 89/111        | 0.137         |
| 64  | 1 (original)      | 138.14       | 129.79     | 8.35          | [3.89, 12.62]    | 8.03       | 138/62        | < 0.001       |
| 64  | 2 (signed)        | 135.37       | 136.63     | −1.26         | [−7.29, 4.06]    | 3.09       | 123/77        | 0.001         |
| 64  | pooled            | 136.75       | 133.21     | 3.54 (2.6%)   | [−0.57, 7.46]    | −          | 128/72        | < 0.001       |
| 100 | 1 (original)      | 163.89       | 157.76     | 6.13          | [2.00, 10.13]    | 5.66       | 149/51        | < 0.001       |
| 100 | 2 (signed)        | 164.04       | 162.71     | 1.33          | [−3.63, 6.02]    | 1.51       | 128/72        | < 0.001       |
| 100 | pooled            | 163.97       | 160.24     | 3.73 (2.3%)   | [0.10, 7.20]     | −          | 149/51        | < 0.001       |

(3) Why the whole-request gain is smaller than the removed work.

The removed rewrite is worth 0.59 s per question at concurrency 4 and tens of seconds per branch under load, yet the response-time saving is 0.1–3.7 s. Table 4 shows where the time goes. From concurrency 32 the orchestrator engine reaches 100% KV usage with preemptions, and its queue time per request rises from 0.1–0.5 s to 28, 85–87, and 122–138 s. A request that leaves the search branch earlier waits for the orchestrator instead; at concurrency 100 the adapter’s requests even carry more orchestrator queueing than the original’s (138 versus 122 s per request). Throughput and the

share of requests completed within 60 s are equal between arms at every level (93–94% at 8, 61–66% at 16, 15–18% at 32, 8–11% at 64 and 100): the closed loop fixes throughput, and the orchestrator, not the search branch, sets the tail. The adapter therefore removes a real and growing cost on the search branch, and the response-time benefit is bounded by the next bottleneck.

#### 6.4 Quality

(1) Agreement with the original route.

Source agreement uses the same-round original response

as reference. At concurrency 4, signed dispatch reproduces 98.53%, 98.25%, and 98.50% of the reference's law IDs over 173, 174, and 172 nonempty references, and its exact law-set agreement is 182/199, 184/200, and 183/198. Repeated original rounds agree at 98.87%, 98.62%, and 99.13% recall and 190/199, 187/198, and 192/199 exact sets. The same 26 questions return no law in every batch of every condition; they are the questions that never call the search boundary. Under load, agreement with the original falls for both arms as the system saturates: nonempty recall 96% at 8, 95–96% at 16, 94–95% at 32, 88–93% at 64, and 83–87% at 100, while each arm's own round-to-round exact-set agreement falls from 92–94% to 81–83% (original) and 73–82% (signed).

## (2) Is the residual gap caused by dispatch?

Between-arm agreement is a few points below within-arm agreement even at concurrency 4 (89–92% versus 95%), but the two adapter arms, which share one mechanism, also agree with each other at only 91%, and each condition ran in its own application instance. The instance control resolves this. Table 7 shows that two instances of the same route agree on exact law sets in 86.4% (original) and 83.5% (signed) of questions, no more than instances of different routes (83.0–87.0%). For questions whose prepared arguments were identical in both instances, the search handler returned an identical law-ID list in only 65–79% of cases and a byte-identical result in 5–17%. The retrieval path itself varies between instances by more than the between-arm gap. No source-set loss is attributable to the adapter; the adapter's own contribution to variation, the arguments it dispatches, is identical to the original's on every branch.

**Table 7** Instance control at concurrency 4 (200 questions per instance): exact law-set agreement, nonempty-reference recall, and, among questions whose prepared arguments were identical in both instances, the number with identical law-ID lists from the search handler.

| Pair                | Kind       | Exact set (%) | Recall (%) | Same law list |
|---------------------|------------|---------------|------------|---------------|
| orig-a / orig-b     | same route | 86.4          | 96.8       | 111/164       |
| signed-a / signed-b | same route | 83.5          | 97.0       | 114/170       |
| orig-a / signed-a   | different  | 86.4          | 97.0       | 126/166       |
| orig-b / signed-b   | different  | 87.0          | 96.7       | 132/168       |
| orig-a / signed-b   | different  | 83.9          | 95.6       | 117/168       |
| orig-b / signed-a   | different  | 83.0          | 96.5       | 109/168       |

## 6.5 Signing

Unsigned and signed dispatch differ by 0.36 ms per branch at the boundary and by 0.729, −0.049, and −0.043 s in round means at concurrency 4, differences of the same size as the null control. The envelope is therefore neutral for speed; it adds the scope, expiry, and replay checks that the contract test exercises and that a distributed deployment would need.

## 7. Discussion

### (1) What the adapter solves.

The redundant call is gone from every eligible search branch:

648 of 648 rewrites per condition are replaced by a 0.5–0.9 ms check, the tool/answer engine handles 41–46% fewer requests, its queue time per request falls by 29–72% under load, and requests complete their search 0.5 s earlier at concurrency 4 and by seconds under load. These effects are repeatable across rounds and hold at every concurrency tested.

### (2) What it does not solve.

The user-visible response time improves by a few percent, not by the removed work, because this deployment's orchestrator engine saturates from concurrency 32 and absorbs the released time; at concurrency 4 the improvement (0.5%) is below the round-to-round variation of the original route itself. Removing the rewrite is worthwhile as a service-load and search-latency improvement, and its response-time value grows only when the orchestrator has spare capacity. The adapter does not change the number of orchestrator calls, their decoding, or their KV footprint, which the measurements identify as the binding constraint.

### (3) Limits.

Two rounds per sweep level leave position effects of up to 9 s at concurrency 32; ten sweep cells and the instance control ran while one stuck request occupied an orchestrator slot; production shared the engines; the question set is single-turn; and source agreement is measured against the original route, not against expert-labeled answers. The instance control shows that quality comparisons of any dispatch change in this system need several instances per arm or a deterministic retrieval configuration.

## 8. Conclusion

A multi-agent legal search system re-serialized prepared search arguments through a model call before every search. Branch traces showed the call to be a pure relay, identical in its output to its input in 648 of 648 cases, and increasingly expensive under load, from 0.54 s per branch at concurrency 4 to 22 s at concurrency 100. A checked direct-dispatch adapter replaced it with a 0.5–0.9 ms validation and dispatch to the same handler, optionally inside a signed envelope that costs 0.36 ms and changes no result. The tool/answer engine's request count fell by 41–46% and its queue time per request by 29–72%; requests reached the end of their search 0.46–0.64 s earlier at concurrency 4. The end-to-end response time improved by 0.5% at concurrency 4, 1.4–2.9% at 8–16, and 2.3–2.6% at 64–100, with per-question sign tests favoring the adapter at every load except 32, because the orchestrator engine saturates and bounds the gain. Source-set agreement with the original route lies within the retrieval path's own instance-to-instance variation. The adapter is a safe, measurable removal of redundant work whose response-time value depends on the capacity of the remaining orchestration stages.

## References

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, 2020. <https://arxiv.org/abs/2005.11401>
- [2] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proc. ICLR*, 2023. <https://arxiv.org/abs/2210.03629>
- [3] T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. <https://arxiv.org/abs/2302.04761>
- [4] Y. Qin et al., “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs,” in *Proc. ICLR*, 2024. <https://arxiv.org/abs/2307.16789>
- [5] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” in *Proc. COLM*, 2024. <https://arxiv.org/abs/2308.08155>
- [6] S. Kim et al., “An LLM Compiler for Parallel Function Calling,” in *Proc. ICML*, 2024. <https://arxiv.org/abs/2312.04511>
- [7] C. Lin et al., “Parrot: Efficient Serving of LLM-based Applications with Semantic Variable,” in *Proc. USENIX OSDI*, pp. 929–945, 2024. <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>
- [8] M. Luo et al., “Agentix: An Efficient Serving Engine for LLM Agents as General Programs,” in *Proc. USENIX NSDI*, pp. 2443–2459, 2026. <https://www.usenix.org/conference/nsdi26/presentation/luo>
- [9] L. Zheng et al., “SGLang: Efficient Execution of Structured Language Model Programs,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024. doi:10.52202/079017-2000.
- [10] C. Jin et al., “RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation,” arXiv:2404.12457, 2024. <https://arxiv.org/abs/2404.12457>
- [11] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” RFC 2104, 1997. <https://www.rfc-editor.org/rfc/rfc2104>
- [12] A. Birgisson et al., “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud,” in *Proc. NDSS*, 2014. doi:10.14722/ndss.2014.23212.